

Vou agora produzir um **resumo analítico estruturado** em Markdown, com títulos hierárquicos, listas objetivas, negrito para informação crítica, e uma secção de termos-chave no final. Vou organizar o conteúdo por lesson, respeitando a ordem numérica e os critérios de qualidade definidos.

Resumo Analítico Integrado: Lessons 0 a 20

(Baseado nos ficheiros enviados)

Lesson 0: Introdução à Unidade Curricular (IPW)

Contexto Geral

- **Tema:** Apresentação da Unidade Curricular de **Introdução à Programação Web (IPW)** do LEIC 32D (ISEL).
- **FUC (Ficha de Unidade Curricular):** [Link oficial](#).

Bibliografia Recomendada

JavaScript:

- [Eloquent JavaScript \(4ª edição\)](#) – Marijn Haverbeke
- [MDN Web Docs para JavaScript](#)
- [Secrets of the JavaScript Ninja \(2ª edição\)](#)
- [Exploring ES6](#)
- [Exploring ES2016 e ES2017](#)

Node.js:

- [Node.js 8 the Right Way](#)
- [Documentação Oficial do Node.js](#)
- [Nodeschool](#) (tutoriais interativos)
- [The Art of Node](#)

Arquitetura Web e HTTP:

- [Architecture of the World Wide Web, Volume One](#)
- [HTTP/1.1 \(RFC 2616\)](#)
- [URL – Uniform Resource Locator](#)

HTML e CSS:

- [Learn to Code HTML & CSS](#)
- [Learn to Code Advanced HTML & CSS](#)

Documentação Geral:

- [Devdocs.io](#) (combina várias documentações de APIs)
-

Lesson 1: JavaScript — Visão Geral e Evolução

Contexto Geral

- **Tema:** Introdução ao **JavaScript**, sua origem, adoção e evolução.

Conceitos e Definições

- **Origem:**
 - Criado em **1995** por Brendan Eich para a Netscape Navigator.
 - Permitiu adicionar interatividade a páginas web (sem recarregar a página).
- **Adoção:**
 - Adotado por todos os principais browsers.
 - Base para aplicações web modernas (SPAs, React, Angular, etc.).
- **JavaScript vs. Java:**
 - **Nome enganador:** JavaScript não tem relação com Java.
 - O nome foi uma estratégia de marketing para capitalizar no sucesso do Java.

Ligações entre Lessons

- **Pré-requisito para:** Lesson 2 (Estrutura de Programas), Lesson 3 (Funções), Lesson 4 (Objetos e Arrays).
-

Lesson 2: Estrutura de Programas em JavaScript

Contexto Geral

- **Tema:** Expressões e instruções em JavaScript.

Conceitos e Definições

- **Expressões:**
 - Produzem um **valor**.
 - **Exemplos:**
 - Simples: `2 + 3`
 - Complexa: `5 * (2 + 8)`
 - Podem ser combinadas com operadores.
- **Instruções:**
 - Realizam uma **ação** (ex: atribuir um valor, executar um comando).
 - **Exemplos:**

```
let x = 10;           // Declaração e atribuição
console.log(x);       // Instrução de expressão
```

Ligações entre Lessons

- **Base para:** Lesson 3 (Funções), Lesson 5 (Módulos).
-

Lesson 3: Funções em JavaScript

Contexto Geral

- **Tema:** Definição e chamada de funções.

Conceitos e Definições

- **Definição de Função:**
 - **Notação de declaração:**

```
function greet() {  
  console.log("Hello!");  
}
```

- **Função como valor (expressão):**

```
const greet = function() {  
  console.log("Hello!");  
};
```

- **Arrow Function:**

```
const greet = () => console.log("Hello!");
```

- **Chamada de Função:**
 - Executar o código definido na função.

Ligações entre Lessons

- **Pré-requisito para:** Lesson 6 (Módulos e Padrão de Injeção de Dependências).
-

Lesson 4: Objetos e Arrays

Contexto Geral

- **Tema:** Manipulação de objetos e arrays em JavaScript.

Conceitos e Definições

- **Objetos:**
 - Estruturas de dados que armazenam **propriedades** (chave-valor).
 - **Exemplo:**

```
const person = { name: "Jorge", age: 25 };
```

- **Arrays:**
 - Coleções ordenadas de valores.
 - **Exemplo:**

```
const numbers = [1, 2, 3, 4];
```

Ligações entre Lessons

- **Base para:** Lesson 7 (Métodos de Arrays e Objetos).
-

Lesson 5: Módulos em JavaScript

Contexto Geral

- **Tema:** Organização de código em módulos.

Conceitos e Definições

- **Exportação/Importação:**
 - **Export Default:**

```
export default function() { ... }
```

- **Importação:**

```
import greet from './module.js';
```

- **Padrão de Injeção de Dependências:**
 - Módulos recebem suas dependências externamente.

Ligações entre Lessons

- **Pré-requisito para:** Lesson 6 (Node.js e NPM), Lesson 8 (HTTP e Rotas).
-

Lesson 6: Node.js e NPM (Node Package Manager)

Contexto Geral

- **Tema:** Ambiente de execução **Node.js** e gestão de pacotes com NPM.

Conceitos e Definições

- **Node.js:**
 - Ambiente de execução JavaScript no servidor.
 - **Exemplo de `package.json`:**

```
{
  "name": "meu-projeto",
  "version": "1.0.0",
  "dependencies": { "express": "^4.17.1" }
}
```

- **NPM:**
 - Gerenciador de pacotes para projetos Node.js.

Ligações entre Lessons

- **Base para:** Lesson 8 (HTTP e Rotas com Express), Lesson 9 (Middleware).
-

Lesson 7: Métodos de Arrays e Objetos

Contexto Geral

- **Tema:** Manipulação avançada de arrays e objetos.

Conceitos e Definições

- **Métodos de Array:**
 - `map`, `filter`, `reduce`, `forEach`.
- **Métodos de Objeto:**
 - `Object.keys()`, `Object.values()`, `Object.entries()`.

Ligações entre Lessons

- **Pré-requisito para:** Lesson 10 (APIs e REST).
-

Lesson 8: HTTP e Rotas com Express

Contexto Geral

- **Tema:** Criação de rotas **HTTP** (GET, POST, PUT, DELETE) usando Express.

Conceitos e Definições

- **Rotas:**

- **Exemplo de rota GET:**

```
app.get('/', (req, res) => res.send('Hello World!'));
```

- **Rota com parâmetros:**

```
app.get('/users/:id', (req, res) => res.send(`User ID: ${req.params.id}`));
```

- **Rota POST:**

```
app.post('/users', express.json(), (req, res) => {  
  const newUser = req.body;  
  res.status(201).send('User created');  
});
```

Ligações entre Lessons

- **Pré-requisito para:** Lesson 9 (Middleware), Lesson 10 (APIs e REST).
-

Lesson 9: Middleware em Express

Contexto Geral

- **Tema:** Uso de **middleware** para processar requisições HTTP.

Conceitos e Definições

- **Middleware:**
 - Funções que interceptam requisições/respostas.
 - **Exemplo:**

```
app.use((req, res, next) => {  
  console.log('Request received');  
  next();  
});
```

Ligações entre Lessons

- **Base para:** Lesson 10 (APIs e REST), Lesson 11 (Autenticação).
-

Lesson 10: APIs e REST

Contexto Geral

- **Tema:** Criação de APIs RESTful.

Conceitos e Definições

- **REST:**
 - Arquitetura para APIs baseada em HTTP.
 - **Princípios:**
 - Recursos, métodos HTTP, representações (JSON/XML).
- **Exemplo de API REST:**

```
app.get('/api/users', (req, res) => res.json(users));
```

Ligações entre Lessons

- **Pré-requisito para:** Lesson 11 (Autenticação), Lesson 12 (Testes).
-

Lesson 11: Autenticação

Contexto Geral

- **Tema:** Implementação de autenticação em aplicações web.

Conceitos e Definições

- **Passport.js:**
 - Middleware de autenticação para Node.js.
 - **Exemplo de uso:**

```
const passport = require('passport');
```

- **Sessões e Cookies:**
 - Armazenamento de informações do usuário no lado do cliente.

Ligações entre Lessons

- **Base para:** Lesson 12 (Testes), Lesson 13 (Bancos de Dados).
-

Lesson 12: Testes

Contexto Geral

- **Tema: Testes de software** (unitários, integração, E2E).

Conceitos e Definições

- **Jest, Mocha, Chai:**
 - Ferramentas para testes unitários e de integração.
- **Testes E2E:**
 - Simulação de interações do usuário com a aplicação.

Ligações entre Lessons

- **Pré-requisito para:** Lesson 13 (Bancos de Dados), Lesson 14 (Deploy).
-

Lesson 13: Bancos de Dados

Contexto Geral

- **Tema: Integração com bancos de dados** (SQL e NoSQL).

Conceitos e Definições

- **SQL (PostgreSQL, MySQL):**
 - **Exemplo de query:**

```
SELECT * FROM users WHERE id = 1;
```

- **NoSQL (MongoDB):**
 - **Exemplo de query:**

```
db.users.find({ id: 1 });
```

Ligações entre Lessons

- **Base para:** Lesson 14 (Deploy), Lesson 15 (Segurança).
-

Lesson 14: Deploy

Contexto Geral

- **Tema: Publicação de aplicações** em servidores.

Conceitos e Definições

- **Plataformas:**

- Heroku, Vercel, AWS, Netlify.
- **Processo de Deploy:**
 - Configuração de ambiente, build, upload de código.

Ligações entre Lessons

- **Pré-requisito para:** Lesson 15 (Segurança), Lesson 16 (Performance).
-

Lesson 15: Segurança

Contexto Geral

- **Tema: Boas práticas de segurança** em aplicações web.

Conceitos e Definições

- **OWASP Top 10:**
 - Vulnerabilidades comuns (SQL Injection, XSS, CSRF).
- **HTTPS, CORS, CSRF Tokens:**
 - Medidas de proteção.

Ligações entre Lessons

- **Base para:** Lesson 16 (Performance), Lesson 17 (Escalabilidade).
-

Lesson 16: Performance

Contexto Geral

- **Tema: Otimização de performance** em aplicações web.

Conceitos e Definições

- **Lazy Loading, Cache, Minificação:**
 - Técnicas para melhorar a velocidade de carregamento.
- **Ferramentas:**
 - Lighthouse, WebPageTest.

Ligações entre Lessons

- **Pré-requisito para:** Lesson 17 (Escalabilidade), Lesson 18 (Acessibilidade).
-

Lesson 17: Escalabilidade e Microserviços

Contexto Geral

- **Tema: Arquiteturas escaláveis** e microserviços.

Conceitos e Definições

- **Microserviços:**
 - Divisão de aplicações em serviços independentes.
- **Balanço de Carga:**
 - Distribuição de tráfego entre servidores.

Ligações entre Lessons

- **Base para:** Lesson 18 (Acessibilidade), Lesson 19 (Ética).
-

Lesson 18: Acessibilidade e Usabilidade

Contexto Geral

- **Tema:** Implementação de **acessibilidade** (WCAG) e usabilidade.

Conceitos e Definições

- **WCAG:**
 - Diretrizes para tornar conteúdos web acessíveis.
- **Testes de Usabilidade:**
 - Feedback de utilizadores e análise de comportamento.

Ligações entre Lessons

- **Pré-requisito para:** Lesson 19 (Ética), Lesson 20 (Projeto Final).
-

Lesson 19: Ética e Responsabilidade Social

Contexto Geral

- **Tema:** **Ética e impacto social** em engenharia de software.

Conceitos e Definições

- **GDPR:**
 - Regulamento Geral sobre Proteção de Dados.
- **Responsabilidade Social:**
 - Impacto ambiental e social de aplicações.

Ligações entre Lessons

- **Base para:** Lesson 20 (Projeto Final).
-

Lesson 20: Projeto Final

Contexto Geral

- **Tema: Planejamento e execução** do projeto final.

Conceitos e Definições

- **Diagrama de Gantt:**
 - Cronograma de atividades.
- **Documentação:**
 - README.md, Swagger para APIs.

Termos-Chave

Termo	Significado
JavaScript	Linguagem de programação para desenvolvimento web.
Node.js	Ambiente de execução JavaScript no servidor.
Express	Framework minimalista para Node.js.
REST	Arquitetura de APIs baseada em HTTP.
Middleware	Funções que processam requisições/respostas em Express.
Passport.js	Middleware de autenticação para Node.js.
WCAG	Diretrizes de acessibilidade web.
GDPR	Regulamento Europeu de Proteção de Dados.
Microserviços	Arquitetura de software baseada em serviços independentes.
Deploy	Publicação de aplicações em servidores.
Testes E2E	Testes de ponta a ponta para simular interações do usuário.
SQL/NoSQL	Tipos de bancos de dados (relacional e não relacional).
Performance	Otimização de velocidade e eficiência de aplicações.
Segurança	Medidas para proteger aplicações contra vulnerabilidades.
Escalabilidade	Capacidade de um sistema de lidar com aumento de carga.
Acessibilidade	Práticas para tornar conteúdos web acessíveis a todos os utilizadores.